raft-java Developer Cheatsheet

Felix Berger

June 2026

Quick Start

```
// 1. Build
./gradlew build

// 2. Run 3-node cluster
java -jar build/libs/raft-java-1.0.0-all.jar \
  config/node1.properties
```

Configuration (.properties)

```
# Required
node.id=node1
node.port=9091
data.dir=/var/raft/node1
peer.node1=host1:9091
peer.node2=host2:9092
peer.node3=host3:9093

# Optional (defaults shown)
snapshot.threshold=100
snapshot.chunk.size=1048576
metrics.port=0
```

Custom State Machine

```
public class MyStateMachine
    implements StateMachine {

  @Override
  public byte[] apply(byte[] command) {
    // Apply committed command
    // Return result bytes to client
    return "OK".getBytes();
  }

  @Override
  public byte[] takeSnapshot() {
    // Serialize entire state
    return serialize(state);
  }

  @Override
  public void restoreSnapshot(byte[] data) {
    // Deserialize and replace state
    state = deserialize(data);
```

```
  }
}
```

Register as a Spring bean to override the default `KeyValueStateMachine`:

```
@Bean
public StateMachine stateMachine() {
  return new MyStateMachine();
}
```

Submitting Commands

In-process (server-side):

```
RaftNode node = ctx.getBean(RaftNode.class);

CompletableFuture<byte[]> f =
    node.submitCommand(cmd);
byte[] result = f.get(2, SECONDS);
```

Over gRPC (client-side):

```
RaftClient client = new RaftClient(
    Map.of("n1","host1:9091",
           "n2","host2:9092",
           "n3","host3:9093"));
byte[] result = client.submit(cmd);
```

Cluster Reconfiguration (§6)

```
// Add a member (leader only)
node.addServer("n4", "host4:9094")
    .get(2, SECONDS);

// Remove a member (leader only)
node.removeServer("n4")
    .get(2, SECONDS);
```

One change at a time. Wait for commit before the next.

Transport Layer

Switch to Netty TCP (instead of gRPC):

```
@Bean
public RaftTransportFactory transportFactory() {
  return new NettyTransportFactory();
}
```

Custom transport:

```
public class MyTransport
    implements RaftTransport {

  CompletableFuture<RequestVoteResponse>
    requestVote(RequestVoteRequest req);

  CompletableFuture<AppendEntriesResponse>
    appendEntries(AppendEntriesRequest req);

  CompletableFuture<InstallSnapshotResponse>
    installSnapshot(InstallSnapshotRequest r);
```

```
  CompletableFuture<PreVoteResponse>
    preVote(PreVoteRequest req);

  void close();
}
```

Key Interfaces

Interface	Purpose
`StateMachine`	Your app logic
`RaftStorage`	Durable state
`RaftTransport`	Peer comms
`RaftTransportFactory`	Connection factory
`RaftRpcHandler`	Incoming RPCs

Storage Implementations

Class	Use
`BerkeleyDbStorage`	Production (fsync)
`InMemoryStorage`	Tests only

Node Lifecycle

```
// Spring wiring (production)
System.setProperty("raft.config.path",
    "config/node1.properties");
var ctx = new AnnotationConfigApplicationContext(
    RaftNodeConfiguration.class);
RaftNode node = ctx.getBean(RaftNode.class);
node.start();
// ... use node ...
ctx.close(); // orderly shutdown

// Manual wiring (tests)
RaftNode node = new RaftNode(
    config, storage, stateMachine,
    transportFactory, RaftMetrics.noop());
node.start();
node.shutdown();
```

Testing

```
// Single-node (instant leader)
RaftConfig cfg = RaftConfig.load(tmpFile);
RaftNode node = new RaftNode(cfg,
    new InMemoryStorage(),
    new KeyValueStateMachine(),
    addr -> null,  // no peers
    RaftMetrics.noop());
node.start(); // immediately leader

// In-process gRPC (multi-node)
ManagedChannel ch = InProcessChannelBuilder
```

```
    .forName(addr).directExecutor().build();
RaftTransport t = new GrpcTransport(ch);

Server srv = InProcessServerBuilder
    .forName(addr).directExecutor()
    .addService(new GrpcTransportServer
        .RaftServiceAdapter(node))
    .build().start();
```

Metrics (Prometheus)

`curl http://localhost:10091/metrics`

Counters: `raft.election.started`, `.won` | `raft.vote.granted` | `raft.stepdown` | `raft.entry.applied` | `raft.replication.sent`, `.success`, `.failure` | `raft.snapshot.taken`, `.installed`, `.chunk.sent`, `.chunk.received` | `raft.client.rejected`

Timers: `raft.rpc.append.entries`, `.request.vote`, `.install.snapshot` | `raft.client.submit`

Gauges: `raft.node.term`, `.commit.index`, `.last.applied`, `.role`, `.log.last.index`, `.snapshot.index` | `raft.cluster.size`

Proto RPCs (raft.proto)

RPC	Direction	Purpose
`RequestVote`	peer→peer	Election
`AppendEntries`	leader→follower	Replication + heartbeat
`InstallSnapshot`	leader→follower	Catch-up
`PreVote`	candidate→peer	Pre-election check

Client RPCs in `client.proto`: `Submit`, `AddServer`, `RemoveServer`

Performance Tuning

Parameter	Default	Effect
`snapshot.threshold`	100	Entries before auto-snapshot
`snapshot.chunk.size`	1 MB	InstallSnapshot chunk size
`HEARTBEAT_INTERVAL_MS`	50	Heartbeat frequency
`ELECTION_TIMEOUT_*_MS`	150-300	Election timeout range
`MAX_BATCH_BYTES`	1 MB	Max AppendEntries size
`MAX_INFLIGHT_APPENDS`	2	Pipeline depth per peer